Running zgz-pileup on RUN1, step by step

What each script prints and draws, from a complete run of the package on the three channels of RUN1

This is a record of one full run of the package, from the downloaded repository to the result tables, as you would see it on your screen. Each step shows the command, what the script printed (long progress counters are cut to their first and last line), how long it took, and the figures it drew, with what to look at in them. The figures are those of channel 1; channels 0 and 2 have the same figures in their own folders. Paths are shown as the example paths of `config.py`. The numbers are the same as in the report: the run reproduces it.

0. Before you start

Get the code and make the Python environment (usage guide, section 1):

```
$ git clone <your fork of zgz-pileup>
$ cd zgz-pileup
$ conda env create -f environment.yml
$ conda activate zgz-pileup
```

The data of the run, as the acquisition wrote it, in one folder:

```
$ ls -s /data/ZGZ/RUN1
   6.7 GB  c0.zip
   5.9 GB  c1.zip
   6.5 GB  c2.zip
   319 MB  out.txt
```

In `config.py`, point `RUNS` at that folder and `WORK` at a folder with a few GB free; everything else keeps its default:

```
RUNS = {"RUN1": "/data/ZGZ/RUN1"}
CHANNELS = [0, 1, 2]
WORK = "/data/ZGZ/zgz-work"
POOL_RUNS = True
PEAK_KEV = 766
WORKERS = 3
MAX_TEMP = 85
```

1. Read the records and give them a time

Reads every record of `c<ch>.zip` straight from the zip into a compact file, detects the sampling period and the ADC step, and matches each record to its row in `out.txt`, which gives it its time. This is the slowest step after the fits: it reads about 25 GB of text per channel.

```
$ python scripts/01_read_data.py
```

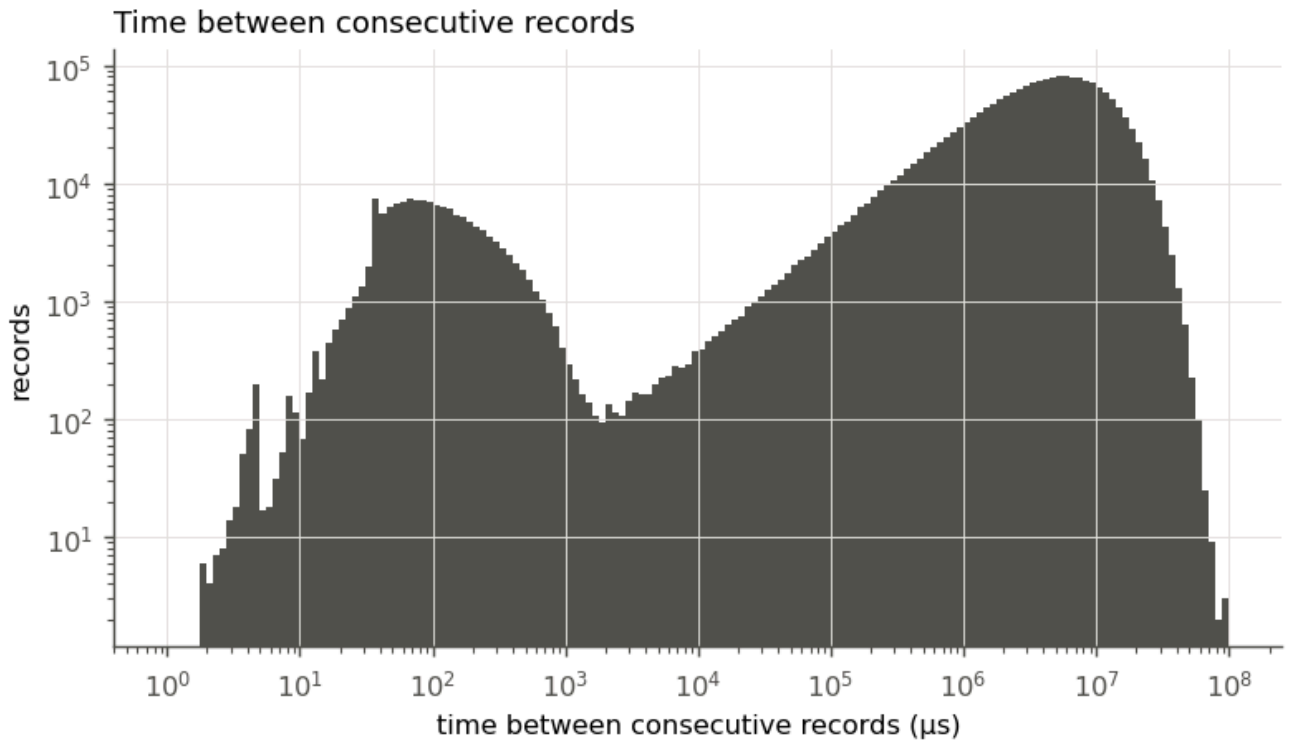
```
=== RUN1 channel 0 ===
reading /data/ZGZ/RUN1/c0.zip
        5 M lines, 13,343 records,     2 s
  …
      827 M lines, 2,611,404 records,   210 s
2,611,405 records, 824,220,458 samples; sampling 128.0 ns; ADC step 257.07; 211 s
2,611,405 records; 2,611,317 matched to out.txt (100.00%); 3.6 min

=== RUN1 channel 1 ===
reading /data/ZGZ/RUN1/c1.zip
        5 M lines, 13,916 records,     1 s
  …
      733 M lines, 2,036,168 records,   178 s
2,036,169 records, 731,406,809 samples; sampling 128.0 ns; ADC step 254.42; 179 s
2,036,169 records; 2,036,073 matched to out.txt (100.00%); 3.1 min

=== RUN1 channel 2 ===
reading /data/ZGZ/RUN1/c2.zip
        5 M lines, 13,529 records,     1 s
  …
      804 M lines, 2,469,207 records,   185 s
2,469,207 records, 801,851,403 samples; sampling 128.0 ns; ADC step 256.05; 185 s
2,469,207 records; 2,469,112 matched to out.txt (100.00%); 3.3 min
```

Took 10 min for the three channels.

What to check: the last line of each channel. Practically all records are matched (the 90 or so records per channel without a row are the last ones of the run, which `out.txt` does not have). A low fraction would mean that `out.txt` belongs to another run.



01_record_gaps.png (run folder). Time between consecutive records. The right-hand hump is the ordinary random arrival of records; the left-hand one, from a few µs to about 1 ms, is the bursts in which records follow each other closely. Those bursts are what the tail measurement of step 3 uses.

2. Sort the records and learn the pulse shape

Puts every record in one class (clean single pulse, on the tail of an earlier pulse, saturated, no pulse, pile-up found by a second trigger or by a slow rise, or above the area band), then learns the mean pulse shape from the clean pulses and measures their amplitudes.

```
$ python scripts/02_clean_pulses.py
```

```
=== RUN1 channel 0: classes ===
  features 500,000/2,611,405
  …
  features 2,500,000/2,611,405
classes: {'clean': 2407593, 'baseline': 96852, 'saturated': 14863, 'no pulse': 106, '2nd trigger': 63777, 'slow rise': 27755, 'off diagonal': 459}

=== channel 0: pulse shape from RUN1 ===
photopeak (template amplitude, ADC) per run: RUN1 33.64
shape: t50 at sample 22; {'rise_10_90': 10, 't10_minus_t50': -3, 'D': 28, 'decay10_from_peak': 338}; baseline noise 0.55 ADC

=== RUN1 channel 1: classes ===
  features 500,000/2,036,169
  …
  features 2,000,000/2,036,169
classes: {'clean': 1792079, 'baseline': 129589, 'saturated': 5402, 'no pulse': 367, '2nd trigger': 77648, 'slow rise': 30451, 'off diagonal': 633}

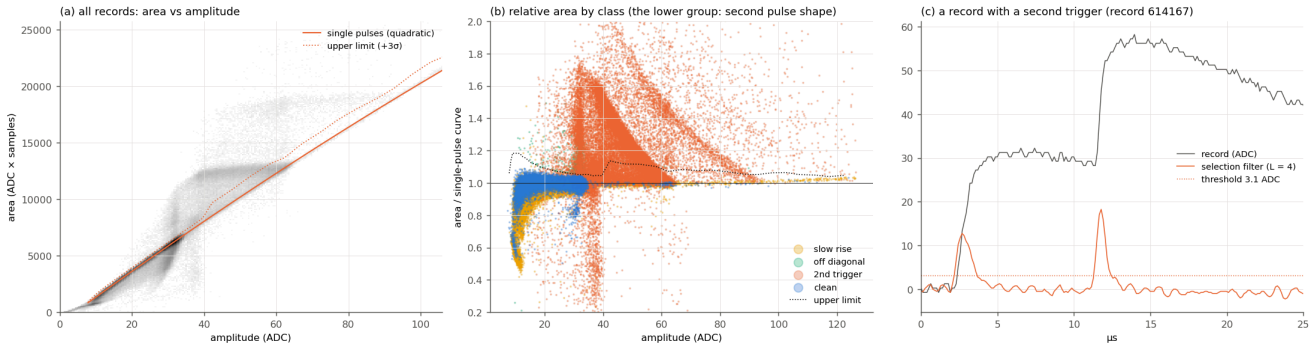
=== channel 1: pulse shape from RUN1 ===
photopeak (template amplitude, ADC) per run: RUN1 31.22
shape: t50 at sample 22; {'rise_10_90': 11, 't10_minus_t50': -4, 'D': 27, 'decay10_from_peak': 344}; baseline noise 0.60 ADC

=== RUN1 channel 2: classes ===
  features 500,000/2,469,207
  …
  features 2,000,000/2,469,207
classes: {'clean': 2281258, 'baseline': 94548, 'saturated': 8568, 'no pulse': 102, '2nd trigger': 72015, 'slow rise': 12037, 'off diagonal': 679}

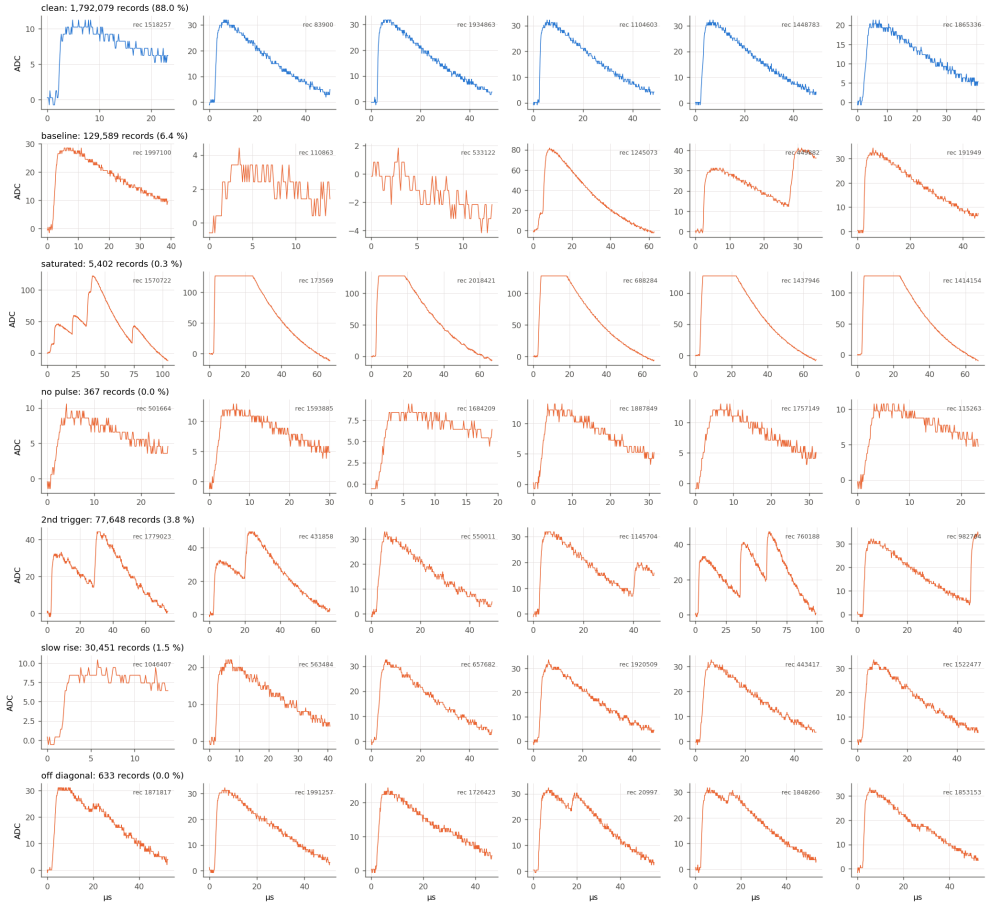
=== channel 2: pulse shape from RUN1 ===
photopeak (template amplitude, ADC) per run: RUN1 31.68
shape: t50 at sample 22; {'rise_10_90': 10, 't10_minus_t50': -3, 'D': 26, 'decay10_from_peak': 344}; baseline noise 0.50 ADC
```

Took 4 min for the three channels.

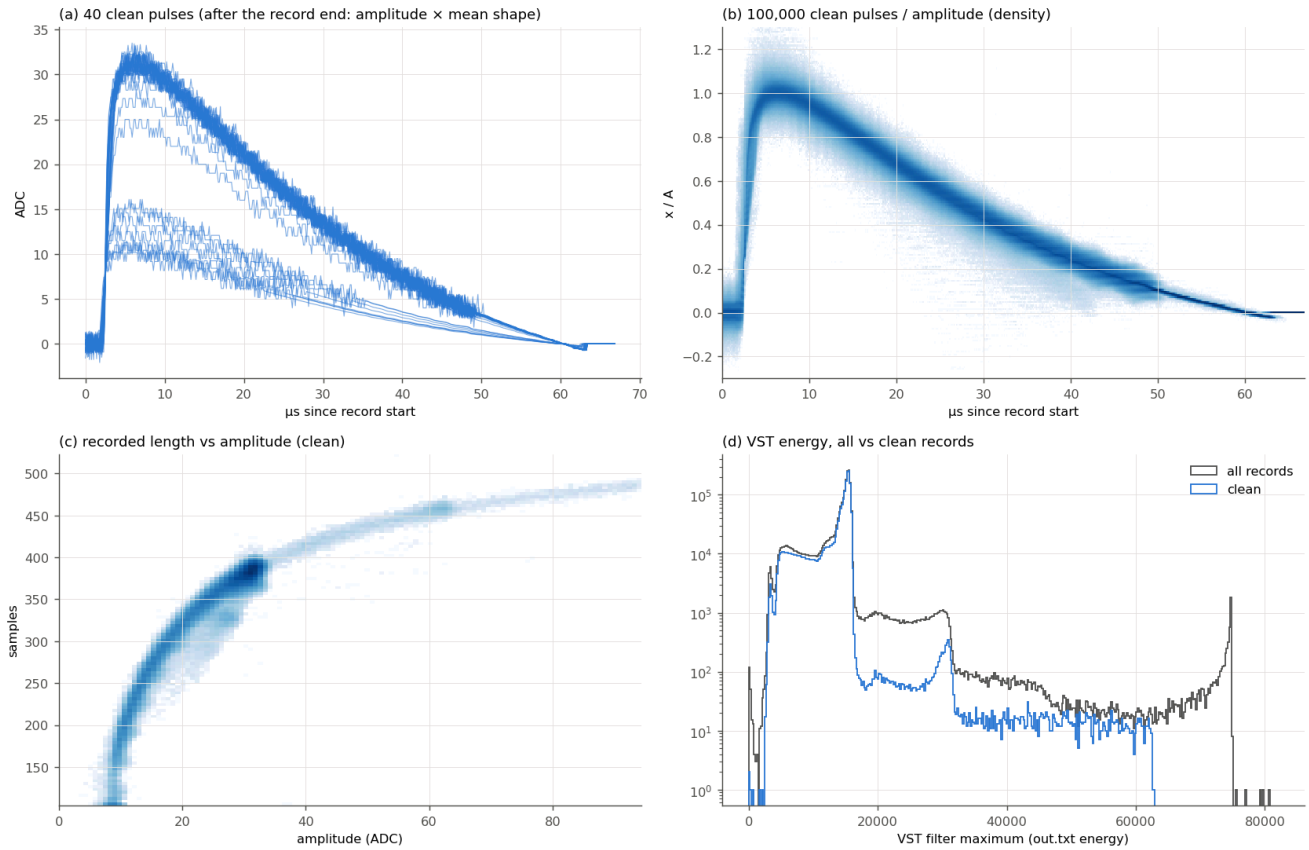
What to check: the class counts (clean pulses are about 88–92 % of the records) and the photopeak amplitude per run, which with several runs tells you whether the gain stayed the same.



02_clean_selection.png (run folder). (a) Area against amplitude of all records: single pulses lie on the solid curve, pile-up adds area and lies above the dotted limit. (b) Area relative to that curve, by class: clean pulses (blue) at 1, and a lower group of clean pulses at 0.6–0.85, the second, faster-decaying pulse shape. (c) A record rejected by the short selection filter, which crosses its threshold once on each rise.



03_record_classes.png (run folder). Six random records of each class. The clean ones should all be single pulses and the pile-up classes should really contain pile-up; *baseline* records start on the tail of an earlier pulse.



04_clean_pulses.png (model folder). (a) Clean pulses, continued after the end of their record with the mean shape; (b) 100,000 clean pulses divided by their amplitude, which should form one narrow band; (c) record length against amplitude: small pulses end their record earlier; (d) the energy of `out.txt` for all records and for the clean ones.

3. Measure the tail and the undershoot

The records stop before the pulse is over, so the end of the pulse is measured from the baseline of the next record whenever a record starts shortly after an isolated clean pulse. Channel 0 prints a warning: its longest record is longer than the measured curve, which is then continued with an exponential, as intended.

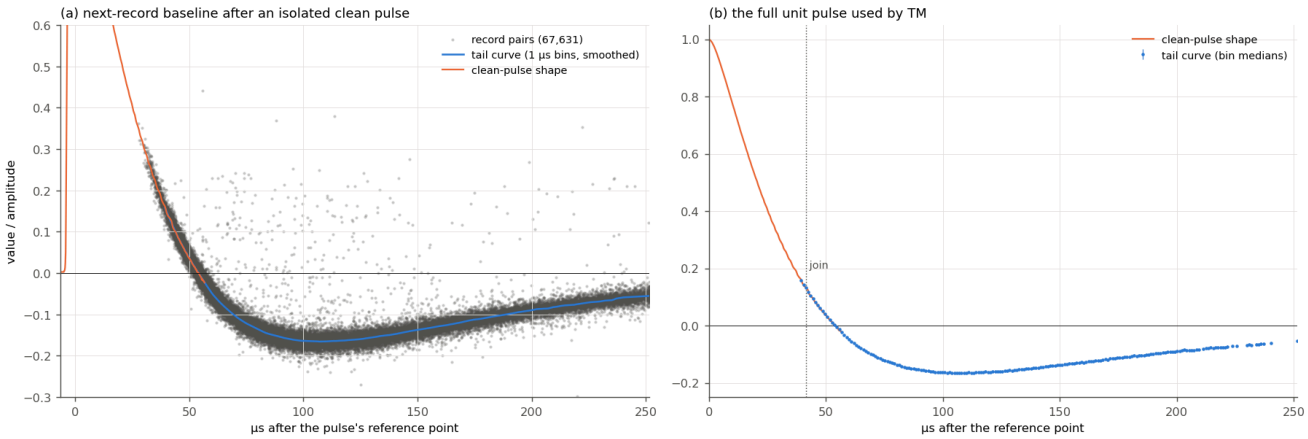
```
$ python scripts/03_tail.py
```

```
=== channel 0: tail from RUN1 ===
RUN1: 59,824 record pairs, baseline zero point 0.42 ADC
warning: tail curve ends at 229 µs, before the longest record (356 µs): extended with the exponential decay of its last 50 µs
tail: zero crossing 52.7 µs, minimum -0.174 of the amplitude at 107 µs

=== channel 1: tail from RUN1 ===
RUN1: 67,631 record pairs, baseline zero point -0.17 ADC
tail: zero crossing 54.8 µs, minimum -0.166 of the amplitude at 107 µs

=== channel 2: tail from RUN1 ===
RUN1: 69,565 record pairs, baseline zero point 0.33 ADC
tail: zero crossing 54.8 µs, minimum -0.164 of the amplitude at 110 µs
```

Took 2 s for the three channels.



05_tail_undershoot.png (model folder), as drawn by this step. (a) Each grey point is one pair of records: the baseline of the next record, divided by the amplitude of the earlier pulse, against the time since it. The blue curve is the measured tail; it joins the clean-pulse shape (orange), crosses zero at about 55 µs and goes down to −0.17 at about 107 µs. (b) The unit pulse: clean-pulse shape, then the measured tail.

4. Choose the trigger of template matching

Chooses the fast-filter length and the threshold of the TM trigger from the clean pulses.

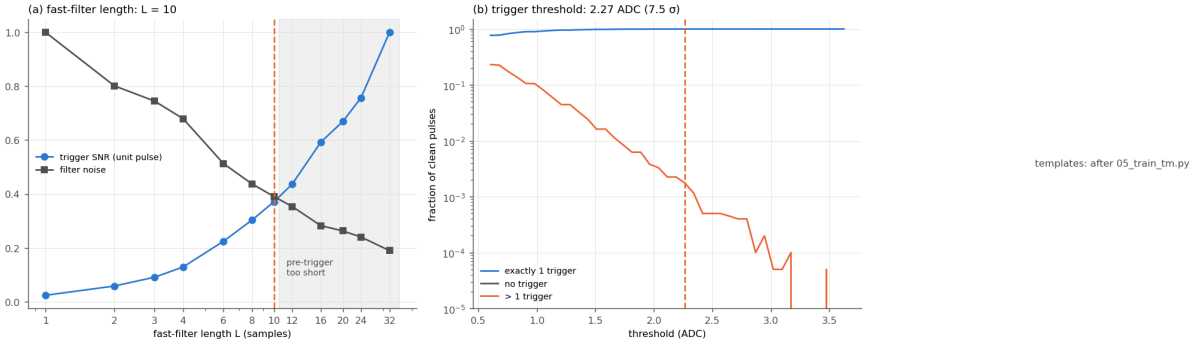
```
$ python scripts/04_tm_params.py
```

```
=== channel 0: TM trigger settings ===
fast filter L = 12 samples, threshold 1.76 ADC (6.8 σ), peak-to-trigger 40 samples

=== channel 1: TM trigger settings ===
fast filter L = 10 samples, threshold 2.27 ADC (7.5 σ), peak-to-trigger 37 samples

=== channel 2: TM trigger settings ===
fast filter L = 10 samples, threshold 1.53 ADC (5.8 σ), peak-to-trigger 36 samples
```

Took 17 s for the three channels.



06_tm_settings.png (model folder), as drawn by this step. (a) Signal-to-noise of the trigger against the filter length; lengths in the grey band need more baseline than the records have. (b) Fraction of clean pulses with exactly one, none or more than one trigger against the threshold; the chosen threshold (dashed) is the lowest at which at most 0.2 % of them trigger twice. Panel (c) is filled in by the next step.

5. Train the templates

Fits the templates on 250,000 clean pulses, joined to the measured tail, and checks them on single pulses held out from the training: the TM amplitude should equal the template

amplitude (the last number, within 0.1 %). It redraws the figures of steps 3 and 4 with the templates.

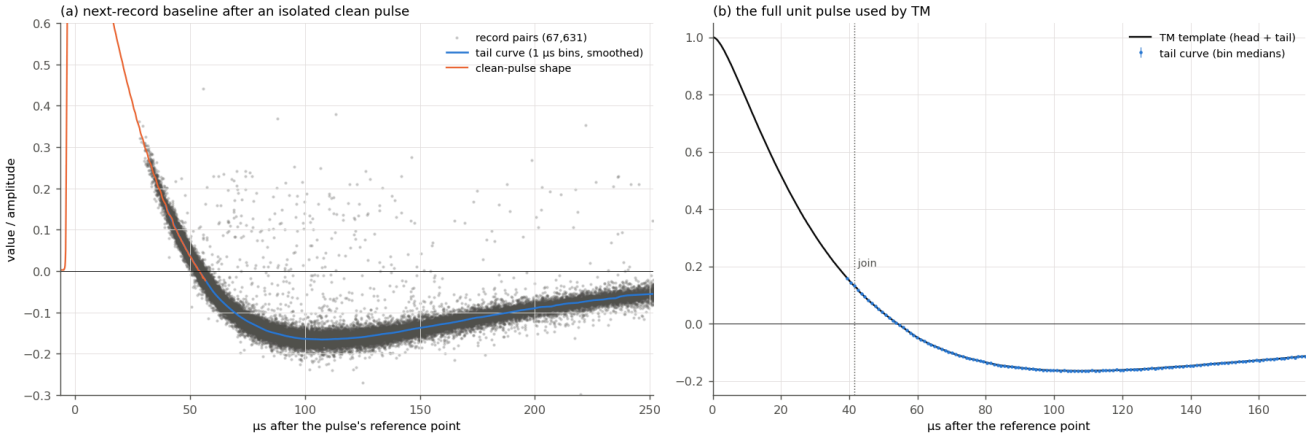
```
$ python scripts/05_train_tm.py
```

```
=== channel 0: TM templates ===
fitting the templates on 250,000 pulses
templates from offset -44 to 2784 samples; residual threshold 6.14 ADC; TM / template amplitude on single pulses 1.0003

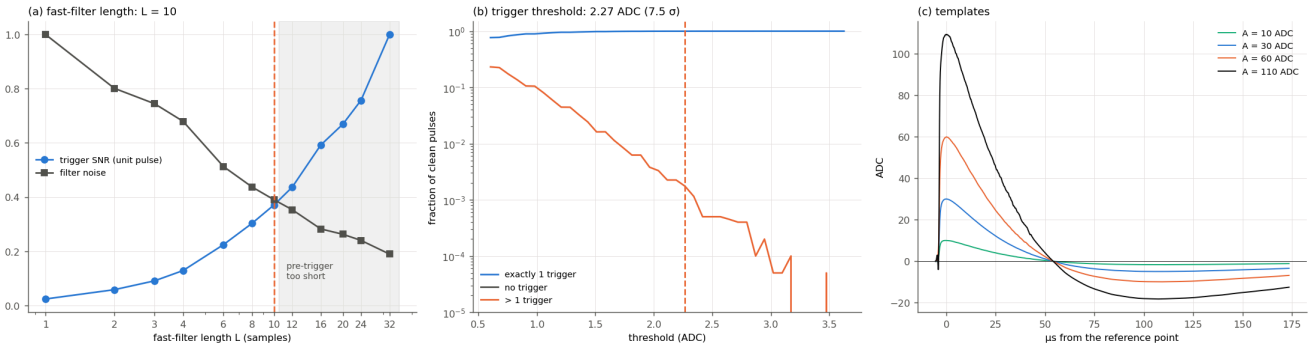
=== channel 1: TM templates ===
fitting the templates on 250,000 pulses
templates from offset -43 to 1356 samples; residual threshold 5.60 ADC; TM / template amplitude on single pulses 1.0001

=== channel 2: TM templates ===
fitting the templates on 250,000 pulses
templates from offset -42 to 1224 samples; residual threshold 5.62 ADC; TM / template amplitude on single pulses 0.9989
```

Took 17 s for the three channels.



05_tail_undershoot.png, redrawn: panel (b) now shows the TM template, which follows the measured tail.



06_tm_settings.png, redrawn: (c) the templates at four amplitudes, with the undershoot.

6. Calibrate the energy

Converts amplitude to energy by putting the photopeak of the clean pulses at `PEAK_KEV`, and finds the sum peak as a check.

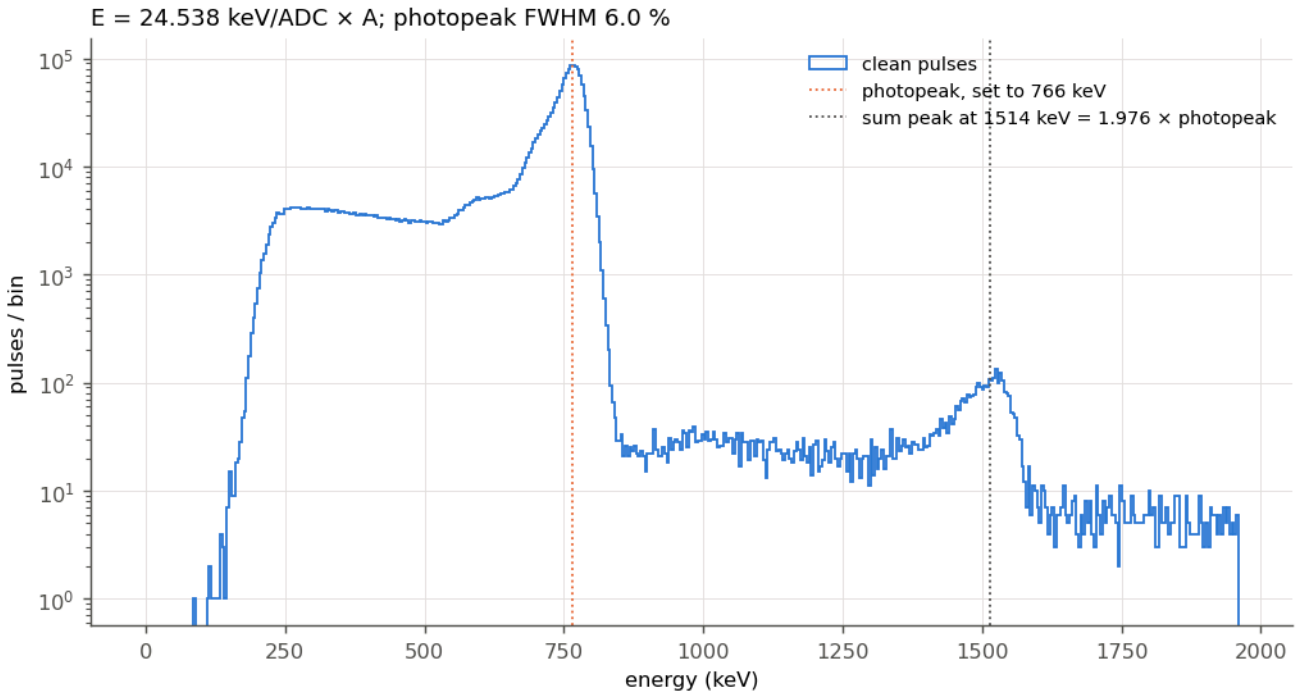
```
$ python scripts/06_calibrate.py
```

```
=== channel 0: energy calibration ===
22.771 keV per ADC; photopeak FWHM 6.1 %; sum peak at 1514 keV (1.977 × photopeak)

=== channel 1: energy calibration ===
24.538 keV per ADC; photopeak FWHM 6.0 %; sum peak at 1514 keV (1.976 × photopeak)

=== channel 2: energy calibration ===
24.181 keV per ADC; photopeak FWHM 5.6 %; sum peak at 1513 keV (1.975 × photopeak)
```

Took 1 s for the three channels.



07_energy_calibration.png (model folder). Spectrum of the clean pulses: the photopeak at 766 keV, which sets the calibration, and the sum peak at 1514 keV, about twice that energy.

7. Benchmark on synthetic pile-up

Builds 5,000 pairs of real clean pulses at known delays, plus 3,000 single pulses, fits them like real records and compares the result with the truth. Each line of the table is one range of delays: the fractions of first, second and both pulses found, and the spread of their energy (%) and time (ns) errors.

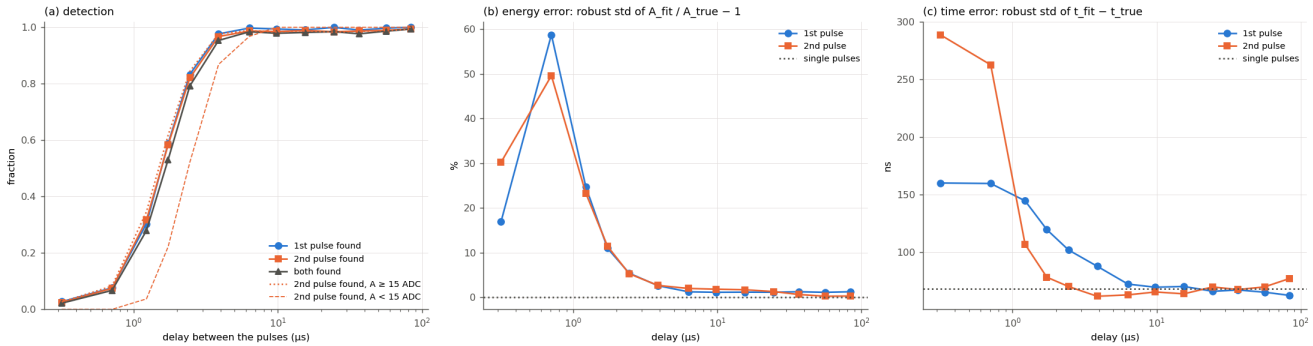
```
$ python scripts/07_benchmark.py
```

```
=== channel 0: benchmark ===
5,000 synthetic pairs + 3,000 single pulses, 3 workers
  fitted 2,000/8,000  0.0 min elapsed, ~0 min left
  …
  fitted 8,000/8,000  0.1 min elapsed, ~0 min left
 delay_us    n  eff_1st  eff_2nd  eff_both  sd_amp_1st_pct  sd_amp_2nd_pct  sd_time_1st_ns  sd_time_2nd_ns  extra_pulses_per_record
    0.316 737    0.007    0.007     0.004          17.889          11.646         172.147         162.793                    0.009
    0.707 550    0.007    0.011     0.004          16.214          21.091         180.644         200.266                    0.013
    1.225 290    0.114    0.124     0.097          62.354          27.589         246.351         141.749                    0.069
    1.732 209    0.407    0.440     0.397          17.979          11.105         173.319         112.349                    0.043
    2.449 318    0.579    0.651     0.550           8.391           6.064         110.667          75.661                    0.022
    3.873 429    0.807    0.867     0.795           3.496           3.375          82.787          67.279                    0.007
    6.325 380    0.971    0.966     0.953           1.621           2.401          72.744          78.518                    0.013
    9.798 333    1.000    0.970     0.970           1.214           1.885          65.596          75.680                    0.003
   15.492 439    0.998    0.982     0.982           1.035           1.587          67.121          76.543                    0.007
   24.495 322    0.997    0.994     0.991           1.210           1.023          67.585          74.892                    0.012
   36.742 305    0.993    0.993     0.987           1.416           0.669          72.101          82.317                    0.010
   56.125 367    1.000    0.992     0.992           1.530           0.345          67.641          70.537                    0.014
   83.666 321    0.997    1.000     0.997           1.721           0.451          68.342          71.563                    0.009
single pulses: {'n': 3000, 'one_pulse_fraction': 0.9956666666666667, 'sd_amp_pct': 0.2390564433190885, 'bias_amp_pct': -0.014811106352230485, 'sd_time_ns': 69.4849119141941, 'bias_time_ns': -0.09923924852319033}
```

```
=== channel 1: benchmark ===
5,000 synthetic pairs + 3,000 single pulses, 3 workers
  fitted 2,000/8,000  0.0 min elapsed, ~0 min left
  …
  fitted 8,000/8,000  0.1 min elapsed, ~0 min left
 delay_us    n  eff_1st  eff_2nd  eff_both  sd_amp_1st_pct  sd_amp_2nd_pct  sd_time_1st_ns  sd_time_2nd_ns  extra_pulses_per_record
    0.316 737    0.027    0.024     0.022          16.940          30.222         160.003         288.860                    0.001
    0.707 550    0.076    0.075     0.067          58.767          49.618         159.572         262.678                    0.002
    1.225 290    0.303    0.317     0.279          24.779          23.338         144.476         106.626                    0.000
    1.732 209    0.589    0.584     0.531          10.996          11.466         119.484          78.094                    0.019
    2.449 318    0.833    0.821     0.792           5.424           5.296         101.774          69.987                    0.009
    3.873 429    0.977    0.967     0.953           2.624           2.685          87.602          61.525                    0.023
    6.325 380    0.997    0.987     0.984           1.251           2.003          72.116          62.727                    0.013
    9.798 333    0.994    0.985     0.979           1.143           1.840          69.371          65.248                    0.027
   15.492 439    0.991    0.989     0.982           1.173           1.718          70.060          63.650                    0.018
   24.495 322    1.000    0.984     0.984           1.154           1.309          65.830          69.638                    0.019
   36.742 305    0.990    0.987     0.977           1.266           0.655          66.981          67.309                    0.030
   56.125 367    0.997    0.989     0.986           1.137           0.255          65.071          69.691                    0.027
   83.666 321    1.000    0.994     0.994           1.266           0.315          62.284          76.974                    0.019
single pulses: {'n': 3000, 'one_pulse_fraction': 0.9916666666666667, 'sd_amp_pct': 0.018125686454698805, 'bias_amp_pct': 0.007622660250494739, 'sd_time_ns': 67.90198130241744, 'bias_time_ns': -1.767938840934221}

=== channel 2: benchmark ===
5,000 synthetic pairs + 3,000 single pulses, 3 workers
  fitted 2,000/8,000  0.0 min elapsed, ~0 min left
  …
  fitted 8,000/8,000  0.1 min elapsed, ~0 min left
 delay_us    n  eff_1st  eff_2nd  eff_both  sd_amp_1st_pct  sd_amp_2nd_pct  sd_time_1st_ns  sd_time_2nd_ns  extra_pulses_per_record
    0.316 737    0.007    0.004     0.001          56.053           5.580         197.342         142.799                    0.001
    0.707 550    0.015    0.025     0.011          67.124          13.061         233.117         142.872                    0.013
    1.225 290    0.159    0.179     0.141          40.952          24.313         180.633         163.202                    0.017
    1.732 209    0.282    0.316     0.268          27.693          15.882         153.472         130.940                    0.005
    2.449 318    0.481    0.560     0.459          11.777           6.150         100.161          73.470                    0.016
    3.873 429    0.746    0.767     0.685           4.782           3.258          74.058          69.923                    0.002
    6.325 380    0.971    0.947     0.926           1.605           1.907          52.484          71.882                    0.018
    9.798 333    0.997    0.964     0.961           1.675           1.920          57.200          67.815                    0.015
   15.492 439    0.998    0.970     0.968           1.199           1.464          67.918          57.181                    0.009
   24.495 322    0.997    0.972     0.969           1.611           1.092          59.329          58.478                    0.022
   36.742 305    0.997    0.993     0.990           2.140           0.689          64.725          67.224                    0.023
   56.125 367    0.995    0.995     0.992           2.126           0.512          55.821          65.157                    0.044
   83.666 321    0.997    0.994     0.994           2.137           0.654          60.320          62.114                    0.019
single pulses: {'n': 3000, 'one_pulse_fraction': 0.9963333333333333, 'sd_amp_pct': 0.3854303639309219, 'bias_amp_pct': -0.0930135902655449, 'sd_time_ns': 61.078844848493674, 'bias_time_ns': -0.13520584799334756}
```

Took 24 s for the three channels.



08_benchmark.png (model folder). (a) Fraction of pulses found against the delay between the two: both are found in more than 95 % of the pairs from about 4 µs on, and pairs closer than about 1 µs are not separated; small second pulses (dashed) need more delay. (b) Energy and (c) time errors; the dotted line is the error on single pulses.

8. Fit every record that is not clean

The long step. Template matching fits every record that is not a clean single pulse (pile-up, records on the tail of an earlier pulse, slow rises), and the script writes the result table `events.csv` with one row per pulse. It prints its progress as it goes, and pauses by itself whenever the CPU is hotter than `MAX_TEMP`. This computer overheated with more processes, so this run used `WORKERS = 1`; with the default of 3 the step takes about a third of the time.

```
$ python scripts/08_recover.py
```

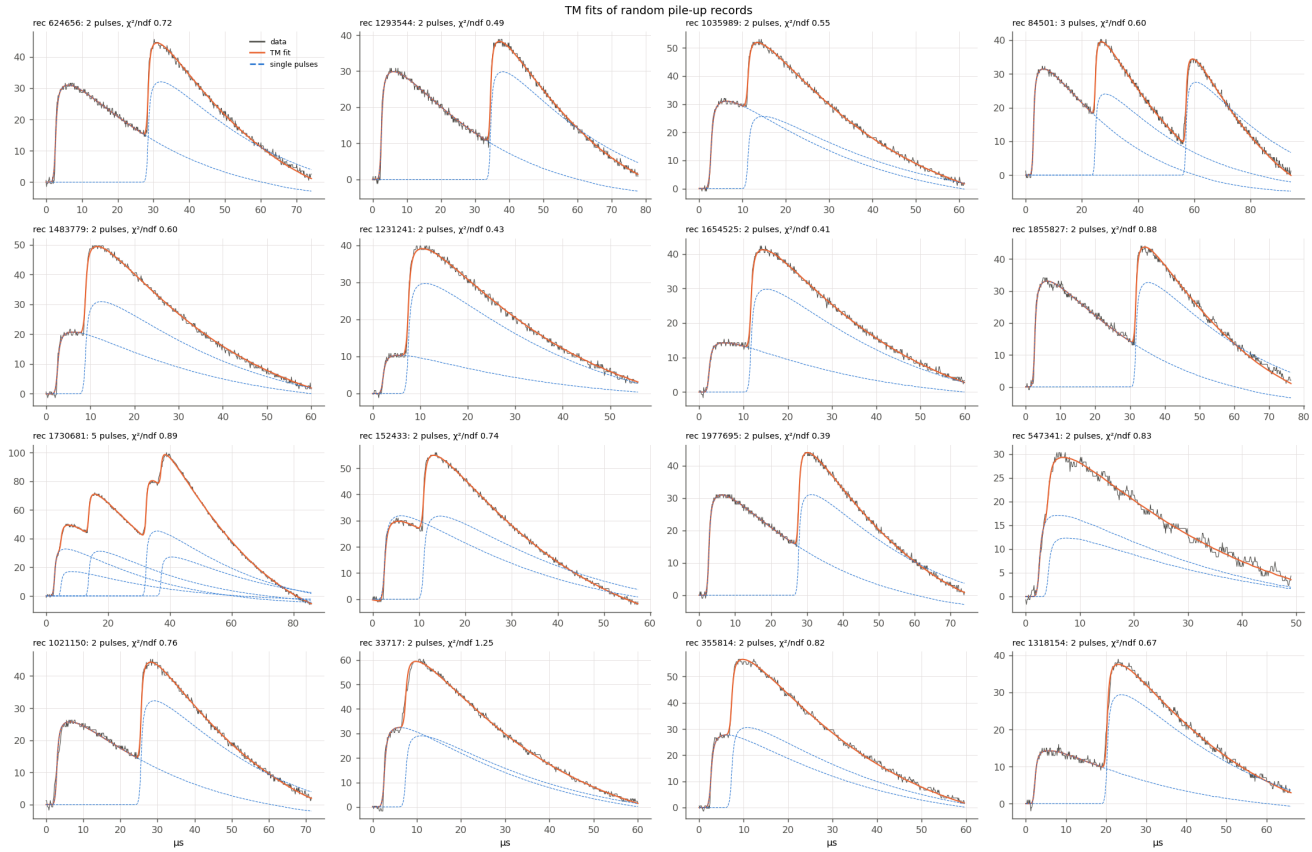
```
=== RUN1 channel 0: TM on the pile-up records ===
TM on 188,843 records, 1 workers
  fitted 2,000/188,843  0.1 min elapsed, ~9 min left
  …
  fitted 188,843/188,843  8.0 min elapsed, ~0 min left
65,760 pile-up records resolved into 139,192 pulses; 2,702,738 rows in events.csv; 8.0 min

=== RUN1 channel 1: TM on the pile-up records ===
TM on 238,321 records, 1 workers
  fitted 2,000/238,321  0.1 min elapsed, ~11 min left
  …
  fitted 238,321/238,321  11.1 min elapsed, ~0 min left
82,554 pile-up records resolved into 178,162 pulses; 2,147,630 rows in events.csv; 11.1 min

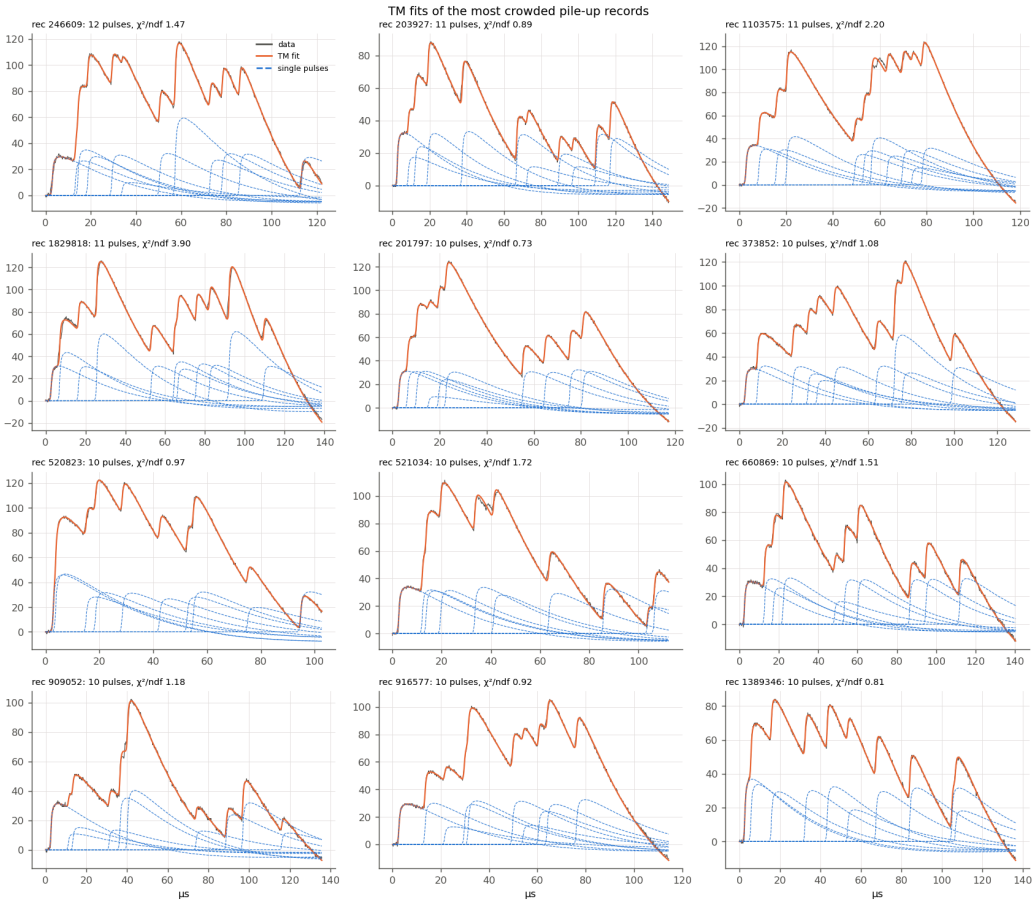
=== RUN1 channel 2: TM on the pile-up records ===
TM on 179,279 records, 1 workers
  fitted 2,000/179,279  0.1 min elapsed, ~11 min left
  …
  fitted 179,279/179,279  12.1 min elapsed, ~0 min left
73,874 pile-up records resolved into 157,455 pulses; 2,572,425 rows in events.csv; 12.2 min
```

Took 32 min for the three channels.

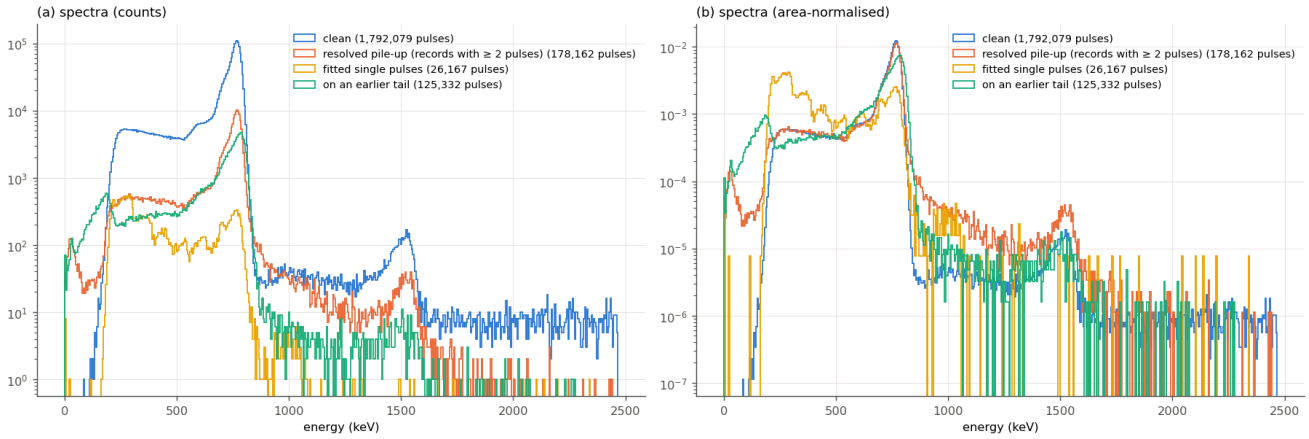
What to check: the last line of each channel: pile-up records resolved, pulses found in them and rows written. For RUN1 they are the numbers of the cover note and the report.



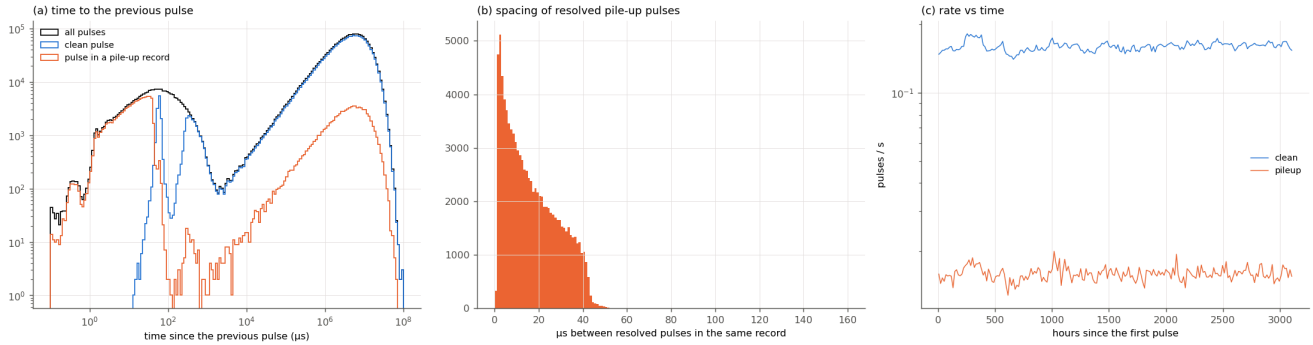
09_pileup_fits_random.png (run folder). Random pile-up records: data, TM fit and the pulses found. The fit should follow the data, rise by rise.



10_pileup_fits_crowded.png (run folder). The most crowded records of the channel, the hardest case.



11_energy_spectra.png (run folder). Energy spectra of clean pulses, of pulses resolved in pile-up records, of single pulses fitted by TM and of pulses on an earlier tail: the photopeak is at the same place, with the same width, for clean and resolved pulses.



12_time_distributions.png (run folder). (a) Time to the previous pulse: the intervals below about 50 µs come from pile-up records and exist only thanks to the fits; (b) spacing of the pulses resolved in the same record; (c) rate of clean and pile-up pulses during the run, which should be steady.

9. Check the results

Three checks on your own data, without using any of our numbers: (1) the pairing of records with `out.txt` rows, showing why it cannot be done row by row (the numbering slips by one row at one or two places per channel, after which naive pairing agrees in energy only by chance); (2) the fitter on records built from held-out clean pulses with known times and amplitudes, true against fitted; (3) the photopeak of clean against resolved pulses.

```
$ python scripts/checks.py
```

```
=== RUN1 channel 0 — check 1: records and out.txt rows ===
records in c0: 2,611,405    rows of channel 0 in out.txt: 2,611,320    difference: +85
record k paired with row k (naive), energy agrees in:
   records          0 —  1,392,885:  91.7%
   records  1,392,886 —  2,201,086:   0.1%
   records  2,201,087 —  2,611,319:   0.0%
paired by step 1 (row = record + offset): 2,611,317 records (99.997%), energy agrees in 91.8%
offset slips after records: [1392885, 2201086] (offsets [0, 1, 3]); records without a row: 88, first at 2611317

=== RUN1 channel 0 — check 2: the fitter on records with known pulses ===
case  true pulses (µs, ADC)                       fitted pulses (µs, ADC)                                  
   1  (6.66, 33.6)                                (6.66, 33.6)                                  χ²/ndf 0.84
   2  (6.53, 33.6), (26.37, 33.6)                 (6.55, 33.4), (26.39, 34.1)                   χ²/ndf 1.25
   3  (6.40, 33.6), (12.54, 16.8)                 (6.40, 33.2), (12.41, 17.4)                   χ²/ndf 1.87
   4  (6.53, 33.6), (9.22, 33.6)                  (6.52, 33.5), (9.29, 33.7)                    χ²/ndf 1.51
   5  (6.53, 33.6), (8.32, 33.6)                  (6.44, 31.9), (8.17, 35.6)                    χ²/ndf 1.40
   6  (6.78, 33.6), (14.46, 20.2), (21.76, 33.6)  (6.72, 33.5), (14.30, 19.5), (21.66, 34.8)   χ²/ndf 1.62
   7  (6.02, 10.1), (10.75, 33.6)                 (6.06, 9.0), (10.77, 35.1)                    χ²/ndf 1.40
   8  (6.40, 33.6), (35.84, 6.7)                  (6.40, 33.5), (35.91, 6.9)                    χ²/ndf 1.11
positions are the reference point (flat-top centre) of each pulse

=== RUN1 channel 0 — check 3: photopeak of clean and of resolved pulses ===
clean                           2,407,593 pulses   photopeak  33.64 ADC   FWHM 6.1 %
resolved pile-up (≥ 2 pulses)     139,192 pulses   photopeak  33.70 ADC   FWHM 6.0 %

=== RUN1 channel 1 — check 1: records and out.txt rows ===
records in c1: 2,036,169    rows of channel 1 in out.txt: 2,036,074    difference: +95
record k paired with row k (naive), energy agrees in:
   records          0 —    334,823:  98.6%
   records    334,824 —  2,036,073:   0.1%
paired by step 1 (row = record + offset): 2,036,073 records (99.995%), energy agrees in 98.7%
offset slips after records: [334823] (offsets [0, 1]); records without a row: 96, first at 2036073

=== RUN1 channel 1 — check 2: the fitter on records with known pulses ===
case  true pulses (µs, ADC)                       fitted pulses (µs, ADC)                                  
   1  (6.27, 31.2)                                (6.33, 31.3)                                  χ²/ndf 0.79
   2  (6.40, 31.2), (26.37, 31.2)                 (6.43, 31.1), (26.34, 31.4)                   χ²/ndf 1.00
   3  (6.27, 31.2), (12.16, 15.6)                 (6.27, 31.2), (12.09, 15.6)                   χ²/ndf 1.58
```

```
   4  (6.40, 31.2), (9.47, 31.2)                  (6.40, 32.5), (9.44, 29.9)                  χ²/ndf 1.18
   5  (6.53, 31.2), (7.81, 31.2)                  (6.36, 27.8), (7.79, 34.6)                  χ²/ndf 1.04
   6  (6.27, 31.2), (14.08, 18.7), (21.12, 31.2)  (6.27, 31.3), (14.10, 18.4), (21.16, 31.3)  χ²/ndf 1.89
   7  (5.50, 9.4), (10.37, 31.2)                  (5.61, 9.4), (10.41, 31.2)                  χ²/ndf 0.84
   8  (6.27, 31.2), (35.20, 6.9)                  (6.27, 31.2), (35.20, 7.1)                  χ²/ndf 0.96
positions are the reference point (flat-top centre) of each pulse

=== RUN1 channel 1 — check 3: photopeak of clean and of resolved pulses ===
clean                            1,792,079 pulses   photopeak  31.22 ADC   FWHM 6.0 %
resolved pile-up (≥ 2 pulses)      178,162 pulses   photopeak  31.28 ADC   FWHM 5.9 %

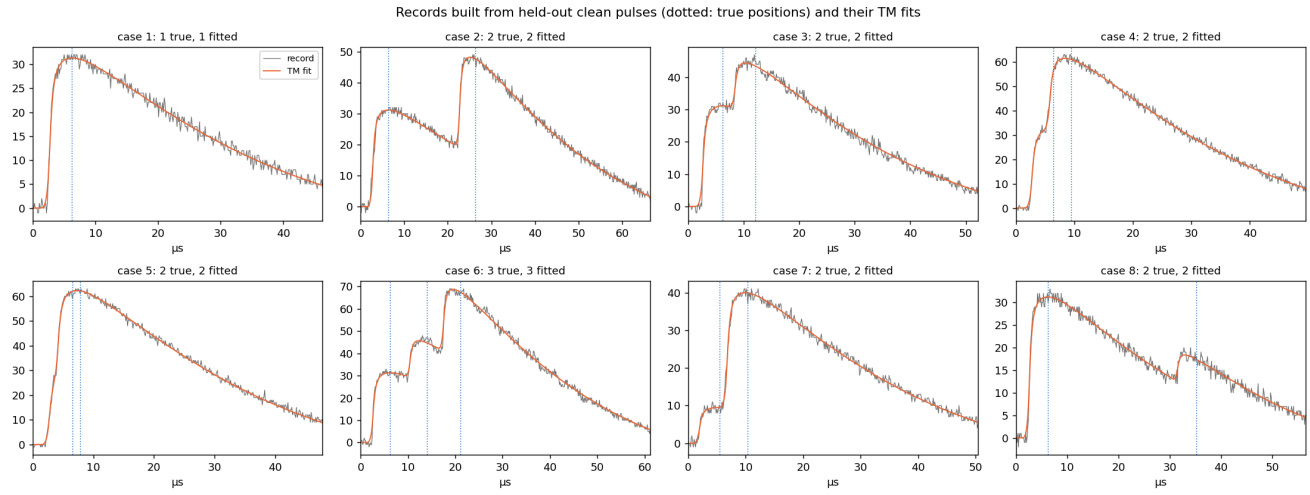
=== RUN1 channel 2 — check 1: records and out.txt rows ===
records in c2: 2,469,207    rows of channel 2 in out.txt: 2,469,113    difference: +94
record k paired with row k (naive), energy agrees in:
   records          0 —    428,969:  99.1%
   records    428,970 —  2,469,112:   0.0%
paired by step 1 (row = record + offset): 2,469,112 records (99.996%), energy agrees in 89.3%
offset slips after records: [428969] (offsets [0, 1]); records without a row: 95, first at 2469112

=== RUN1 channel 2 — check 2: the fitter on records with known pulses ===
case  true pulses (µs, ADC)                      fitted pulses (µs, ADC)
   1  (6.40, 31.7)                               (6.30, 31.7)                                χ²/ndf 0.81
   2  (6.14, 31.7), (26.75, 31.7)                (6.21, 31.4), (26.57, 32.0)                 χ²/ndf 1.47
   3  (6.14, 31.7), (12.03, 15.8)                (6.21, 31.4), (12.13, 16.0)                 χ²/ndf 1.63
   4  (6.40, 31.7), (8.96, 31.7)                 (6.34, 30.1), (9.04, 33.4)                  χ²/ndf 1.17
   5  (6.02, 31.7), (7.94, 31.7)                 (6.16, 33.9), (7.93, 29.4)                  χ²/ndf 1.46
   6  (6.27, 31.7), (14.21, 19.0), (21.38, 31.7)  (6.29, 31.4), (14.18, 18.2), (21.29, 33.1)  χ²/ndf 1.50
   7  (5.63, 9.5), (10.11, 31.7)                 (5.77, 8.1), (10.11, 33.3)                  χ²/ndf 0.99
   8  (6.53, 31.7), (35.71, 6.3)                 (6.39, 31.6), (35.71, 6.6)                  χ²/ndf 1.00
positions are the reference point (flat-top centre) of each pulse

=== RUN1 channel 2 — check 3: photopeak of clean and of resolved pulses ===
clean                            2,281,258 pulses   photopeak  31.68 ADC   FWHM 5.6 %
resolved pile-up (≥ 2 pulses)      157,455 pulses   photopeak  31.70 ADC   FWHM 5.7 %
```

Took 12 s for the three channels.

What to check: in check 1, step 1 pairs practically every record; in check 2, the fitted pulses match the true ones; in check 3, the two photopeaks agree to better than 0.5 % in position and width.



13_checks_fitter.png (model folder). The records of check 2 with their fits; the dotted lines are the true positions. Every case finds the right number of pulses; case 5, two equal pulses 1.3 µs apart, is close to the limit where they can still be separated.

The result

Each channel now has its folder of the run, with the result table `events.csv`, and its folder of the model. For channel 1:

```
$ ls -s ...
/data/ZGZ/zgz-work/runs/RUN1/ch1/
   160 MB  events.csv
    70 MB  pulses.npz
   1.4 GB  records.npz
     2 kB  summary.json
    31 MB  times.npz

/data/ZGZ/zgz-work/models/all/ch1/
     2 kB  benchmark.csv
   449 kB  benchmark.npz
     0 kB  calibration.json
     8 kB  shape.npz
     8 kB  summary.json
     1 MB  tail.npz
    66 kB  tm_model.npz
     3 kB  tm_params.json
     3 kB  tm_settings.json
```

`events.csv` has one row per pulse. Its first rows, and a pile-up record resolved into three pulses, with the main columns (all of them are explained in the usage guide, section 4):

```
 record   kind  n_pulses  pulse      time_s  time_in_record_us  energy_kev flag
      0  clean       1.0      0    3.901341           2.816000  768.274016
      1  clean       1.0      0   12.275113           3.072000  678.758084
      2  clean       1.0      0   15.545296           2.816000  622.502817
      3  clean       1.0      0   27.331813           2.816000  766.069752
    386 fitted       3.0      0 2187.751140           2.701007  769.409202
    386 fitted       3.0      1 2187.751140           4.260913  772.350872
    386 fitted       3.0      2 2187.751150           7.260246  773.066034
```

Time of each step for the three channels on a desktop computer (about 48 minutes in all).

Script	Time
`01_read_data.py`	10 min
`02_clean_pulses.py`	4 min
`03_tail.py`	2 s
`04_tm_params.py`	17 s
`05_train_tm.py`	17 s
`06_calibrate.py`	1 s
`07_benchmark.py`	24 s
`08_recover.py`	32 min
`checks.py`	12 s